

■ Mini-Jogo de Basquetebol 3D

Plano de Desenvolvimento — Three.js + Cannon.js

Introdução à Computação Gráfica · 2025/2026

Visão Geral do Projeto

Jogo de basquetebol 3D em modo treino individual (free-roam). O jogador movimenta-se livremente num pavilhão com campo completo, dribla a bola automaticamente e lança ao cesto mais próximo com uma mecânica de **power bar + zona perfeita** inspirada no NBA 2K. A física de bola, aro e tabela é tratada pelo **Cannon.js**; a renderização e modelação pelo **Three.js**.

Tecnologias	Câmara	Lançamento	Modo de Jogo
Three.js + Cannon.js	3ª Pessoa (follow cam)	Power bar + zona perfeita (auto-aim ao cesto)	Treino livre (sem adversário)

■■ FASE 1 — Setup Base

Tarefa 1.1 — Estrutura do Projeto

- Criar ficheiros index.html, main.js, style.css organizados em pastas (src/, assets/, libs/)
- Importar Three.js r128 e Cannon.js 0.6.2 via CDN ou local (garantir versões compatíveis)
- Configurar renderer WebGL com antialias, tamanho de janela dinâmico e shadow maps activados
- Criar o render loop com requestAnimationFrame e um clock para delta time consistente
- Garantir resize handler para adaptar câmara e renderer ao tamanho da janela

Tarefa 1.2 — Iluminação do Pavilhão

- AmbientLight com intensidade ~0.4 para iluminação base sem sombras duras
 - 4x DirectionalLight posicionadas nos cantos do campo, simulando holofotes de pavilhão desportivo
 - Sombras activadas no renderer (PCFSoftShadowMap) e configuradas em todos os objectos relevantes
 - HemisphereLight opcional para céu/chão, adicionando profundidade visual ao ambiente
- Configurar castShadow e receiveShadow em todos os meshes desde o início para evitar refactor posterior

■■ FASE 2 — O Campo

Tarefa 2.1 — Chão e Envolvente do Pavilhão

- PlaneGeometry para o chão com dimensões reais de campo NBA (28m x 15m, escala 1 unit = 1m)
- Textura de parquet de madeira (MeshStandardMaterial com map, roughnessMap se possível)
- 4 planos como paredes do pavilhão com cor neutra ou textura simples de parede de ginásio
- Tecto opcional com geometria invertida para fechar o espaço visualmente
- BoxGeometry para bancadas esquemáticas nos laterais (blocos simples, sem detalhe)

Tarefa 2.2 — Linhas do Campo

- Técnica: criar um canvas 2D em JavaScript, desenhar todas as linhas nesse canvas, usar como textura por cima do chão (CanvasTexture sobreposta com depthWrite: false)
- Linha central (meia-linha), círculo central com raio correcto (~1.8m)
- Linha de 3 pontos dos dois lados: arco semicircular + duas rectas laterais (raio ~6.75m do centro do cesto)
- Garrafão (zona restritiva) dos dois lados: rectângulo 4.9m x 5.8m
- Linha de lance livre a 4.6m do fundo

- Todas as linhas brancas com espessura visível (~0.05m equivalente)
- Alternativa: usar *LineSegments* do *Three.js* com *BufferGeometry* para cada linha individualmente

Tarefa 2.3 — Tabelas, Postes e Cestos

- Backboard: *BoxGeometry* rectangular (1.83m x 1.07m x 0.05m), material branco semi-transparente ou vidro
- Poste de suporte: *CylinderGeometry* metálica, posicionado atrás e por baixo da tabela
- Braço de suporte horizontal: *BoxGeometry* a ligar poste à tabela
- Aro (rim): *TorusGeometry* (raio ~0.225m, tube ~0.02m), material vermelho-laranja, posicionado 3.05m de altura
- Rede: cone em wireframe ou conjunto de linhas (*LineSegments*) em forma cónica a pender do aro
- Replicar toda a estrutura espelhada no lado oposto do campo
- Posicionar tabelas a 1.22m do fundo do campo (posição oficial)

■ FASE 3 — O Jogador e a Câmara

Tarefa 3.1 — Modelo do Jogador (Primitivos)

- Usar um *Group()* do *Three.js* para agrupar todas as partes do corpo
- Cabeça: *SphereGeometry*, tronco: *BoxGeometry*, braços: *CylinderGeometry* x2, pernas: *CylinderGeometry* x2
- Material simples *MeshStandardMaterial* com cor (não precisa textura), cor de pele para cabeça/mãos
- Camisola com cor configurable, número opcional na frente com canvas texture simples
- Pivot point do grupo no centro dos pés para posicionamento e movimento correctos
- Adicionar um mesh de 'sombra blob' (elipse escura achatada no chão) por baixo do jogador

Tarefa 3.2 — Câmara de 3ª Pessoa

- Câmara *PerspectiveCamera* (FOV ~75, near 0.1, far 500)
- Offset fixo relativo ao jogador: ~3 unidades atrás, ~2 unidades acima no espaço local do jogador
- A câmara segue o jogador com lerp (interpolação linear, factor ~0.1) para movimento suave sem corte brusco
- Quando o jogador roda, a câmara acompanha a rotação com lerp separado no ângulo
- *lookAt()* sempre apontado para a posição do jogador + offset vertical (aponta para o tronco, não para os pés)
- Garantir que câmara não atravessa o chão: clamp na posição Y mínima

Tarefa 3.3 — Movimento do Jogador

- WASD ou setas: movimento relativo à direcção actual do jogador (forward/backward/strafe)
- Q e E (ou rato clicado + arrastar): rotação do jogador à volta do eixo Y
- Velocidade de movimento ~5 m/s, velocidade de rotação ~2 rad/s
- Colisão com as paredes: verificar bounding box do jogador contra os limites do campo a cada frame
- Estado do jogador: enum com IDLE, MOVING, DRIBBLING, SHOOTING, CELEBRATING
- Bloqueio de movimento ao iniciar lançamento (SHOOTING state)
- Não usar física *Cannon.js* para o jogador — movimento controlado directamente por código é mais preciso e simples

■ FASE 4 — Bola e Drible

Tarefa 4.1 — Bola Visual

- *SphereGeometry* (raio 0.12m, 32 segmentos), *MeshStandardMaterial*
- Textura de bola de basquetebol: pode usar textura free (Kenney.nl ou similar) ou gerar com canvas
- Se gerada com canvas: fundo laranja, linhas pretas curvas simulando as costuras da bola
- Adicionar ligeira specular highlight para dar a sensação de bola de borracha
- Modo dribble: bola posicionada ~0.8m à frente e ~0.5m ao lado do jogador (mão direita)

Tarefa 4.2 — Animação de Drible

- Animação controlada por uma função senoidal: $y = \text{height} * |\sin(t * \text{frequency})|$
- Bola sobe até ~0.9m e desce até ~0.05m do chão (altura da bola)
- Easing: usar *Math.abs(Math.sin())* que naturalmente é mais rápido a bater e mais lento no topo

- Frequência dinâmica: quando jogador está parado freq ~1.5Hz, em movimento ~2Hz (dribble mais rápido)
- Animar também a rotação da bola em torno do eixo X para simular a bola a rolar/girar ao quicar
- Acompanhar posição XZ do jogador em tempo real, com pequeno offset de atraso (lerp) para naturalidade
- Dribble PARA quando estado muda para SHOOTING; retoma quando bola volta às mãos

■ FASE 5 — Física com Cannon.js

Tarefa 5.1 — Setup do Mundo Físico

- Criar CANNON.World() com gravidade (0, -9.82, 0)
- Configurar broadphase: NaiveBroadphase ou SAPBroadphase (mais eficiente para muitos objectos estáticos)
- Step do mundo físico no render loop: world.step(1/60, deltaTime, 3) — fixed timestep a 60Hz
- Função syncMeshWithBody(mesh, body): actualiza mesh.position e mesh.quaternion a partir do body Cannon
- Manter uma lista de pares {mesh, body} para sync automático a cada frame

Tarefa 5.2 — Corpos Físicos Estáticos do Campo

- Chão: CANNON.Plane() estático, mass=0, material com friction ~0.4 e restitution ~0.6
- Paredes: CANNON.Box() estáticos para cada parede, mass=0
- Tabela (backboard): CANNON.Box() estático com dimensões exactas do mesh, restitution ~0.5
- Chão do campo: material físico separado com restitution mais alta ~0.75 (madeira de pavilhão)
- Definir CANNON.ContactMaterial entre bola e cada superfície com parâmetros distintos

Tarefa 5.3 — Corpo Físico do Aro (Parte Crítica)

- O aro NÃO pode ser um único Torus — Cannon.js não suporta torus geometry de forma nativa
- Solução: aproximar o aro com 8 a 12 CANNON.Cylinder() pequenos dispostos em círculo
- Cada cilindro tem raio ~0.02m e comprimento ~0.12m (correspondendo ao arco entre segmentos)
- Posicionar e rodar cada cilindro para formar o arco circular completo do aro
- Todos os segmentos são estáticos (mass=0) e agrupados como filho da tabela
- Material do aro: restitution ~0.6 (bola pode ressaltar), friction ~0.3
- Testar múltiplos cenários: swish directo, bate no aro e entra, bate no aro e sai

■ Esta é a tarefa tecnicamente mais complexa da física — testar exhaustivamente antes de avançar

■ Aumentar o número de segmentos (12-16) se a bola 'fantasmar' através do aro

Tarefa 5.4 — Corpo Físico da Bola

- CANNON.Sphere() com raio 0.12m, mass=0.6kg (massa real de bola de basquetebol)
- Material com restitution ~0.75 e linearDamping ~0.3, angularDamping ~0.3
- Durante o dribble: corpo físico DESATIVADO (bola controlada por código/animação)
- No momento do lançamento: corpo físico ACTIVADO com velocidade inicial calculada
- Após a bola parar (velocity < threshold): desactivar corpo físico, retornar bola ao jogador
- Usar body.sleep() / body.wakeUp() para gerir estados activo/inactivo eficientemente

■ FASE 6 — Mecânica de Lançamento (Núcleo do Jogo)

Tarefa 6.1 — Power Bar — Interface Visual

- Elemento HTML/CSS sobreposto ao canvas (position: absolute, pointer-events: none)
- Barra vertical ou horizontal com fundo escuro e fill colorido
- Cor dinâmica: vermelho (0-50%) → amarelo (50-75%) → verde (75-90%) → vermelho outra vez (>90%)
- Zona perfeita destacada: rectângulo semi-transparente verde entre 75% e 88% da barra
- Ícone ou texto 'PERFECT!' que pisca quando a barra está na zona verde
- A barra só aparece quando o jogador entra em SHOOTING state
- Velocidade de enchimento da barra: ~1.2 segundos para encher completamente

Tarefa 6.2 — Auto-Aim e Detecção do Cesto Alvo

- No início do lançamento, calcular distância do jogador aos dois cestos (`Vector3.distanceTo()`)
- O cesto-alvo é sempre o mais próximo — guardar a sua posição (centro do aro) como target
- Calcular vector de direcção horizontal: `direction = (target.xz - player.xz).normalize()`
- A direcção vertical será determinada pelo cálculo balístico (Fase 6.3)
- Opcional: rodar suavemente o jogador para encarar o cesto no início do lançamento (animação de ~0.3s)

Tarefa 6.3 — Cálculo Balístico da Trajectória

- Input: posição do jogador (origem), posição do aro (destino), força da barra (0.0 a 1.0)
- Calcular distância horizontal D e diferença vertical H entre origem e alvo
- Ângulo de lançamento ideal: fixo a 50° (arco bonito e realista para basquetebol)
- Velocidade necessária para ângulo 50°: $v = \sqrt{(g * D^2) / (2 * \cos^2\theta * (D * \tan\theta - H))}$
- Factor de força: $v_final = v_ideal * (0.85 + power * 0.30)$ — power=0 dá menos força, power=1 dá mais
- Zona perfeita (0.75-0.88): $v_final = v_ideal$ com pequeno ruído aleatório $\pm 1\%$ só para realismo
- Fora da zona: desvio crescente até $\pm 15\%$ na força $E \pm 3^\circ$ no ângulo horizontal
- Converter velocidade escalar em vector 3D: $V_x = v * \cos\theta * dirX$, $V_y = v * \sin\theta$, $V_z = v * \cos\theta * dirZ$
- Testar a fórmula balística com valores reais antes de integrar com Cannon.js
- Se usar Cannon.js, aplicar a velocidade calculada directamente: `body.velocity.set(Vx, Vy, Vz)`

Tarefa 6.4 — Animação de Lançamento do Jogador

- Ao largar a tecla de lançamento: transição de estado para SHOOTING
- Animação dos braços: lerp rápido de posição base para posição 'braços levantados' (~0.2s)
- Bola sobe das mãos para a posição de lançamento (~0.3s de animação pre-launch)
- Frame exacto de release: activar corpo físico da bola com velocidade calculada
- Braços regressam à posição base após o lançamento (~0.4s)
- Estado volta a DRIBBLING/IDLE após animação de follow-through

Tarefa 6.5 — Linha de Trajectória Prevista (Opcional, Muito Recomendado)

- Enquanto a barra enche: calcular e mostrar trajectória prevista em tempo real
- Usar Points ou Line do Three.js com 30-50 pontos amostrados ao longo do arco balístico
- Fórmula: para t em [0, t_total], calcular $x(t) = x_0 + V_x * t$, $y(t) = y_0 + V_y * t - 0.5 * g * t^2$
- Cor da linha: verde se dentro da zona perfeita, amarelo/vermelho fora
- Linha desaparece no momento do lançamento (release da tecla)
- A linha é puramente visual — não interage com a física Cannon.js
- Esta funcionalidade melhora imenso o feedback ao jogador e o sentido de skill/precisão

■ FASE 7 — Detecção de Pontos e Scoring

Tarefa 7.1 — Sensor de Cesto (Trigger Zone)

- Criar um CANNON.Cylinder invisível (sem mesh Three.js associado) dentro do aro, ligeiramente menor
- Raio do sensor: ~0.20m (ligeiramente menor que o aro para evitar falsos positivos)
- Usar collision events do Cannon.js: `body.addEventListener('collide', callback)`
- Alternativa mais robusta: verificar a cada frame se a bola cruzou o plano do aro de cima para baixo
- Condição de cesto: posição Y da bola desce abaixo do nível do aro E posição XZ está dentro do raio do aro
- Flag anti-duplo-trigger: após detectar cesto, ignorar colisões por ~0.5s

Tarefa 7.2 — Sistema de Pontuação e Zonas

- Ao disparar o lançamento: registar a posição XZ do jogador como posição_de_lancamento
- Verificar se a posição está dentro ou fora do arco de 3 pontos do cesto-alvo
- Distância ao centro do cesto > 6.75m: regista como potencial 3 pontos
- Distância ao centro do cesto <= 6.75m: regista como potencial 2 pontos
- Pontuar apenas quando a detecção de cesto for confirmada (Tarefa 7.1)
- Manter contadores: total de pontos, cestos de 2, cestos de 3, tentativas totais, percentagem de acerto

- Actualizar o HUD com animação de contador (tween do número de 0 ao novo valor)

Tarefa 7.3 — Feedback Visual e Sonoro

- Texto '+2' ou '+3' a surgir sobre o cesto e a subir/desaparecer (CSS animation ou Three.js sprite)
- Flash de cor no ecrã: overlay semi-transparente verde que aparece e desaparece em ~0.4s
- Animação de celebração do jogador: braços levantados, pequeno salto (deslocamento Y de ~0.2m)
- Som de cesto: Web Audio API com AudioBuffer carregado de ficheiro .mp3 ou sintetizado
- Som de bola a bater no aro: tom metálico curto reproduzido na colisão com os segmentos do aro
- Som de drible: som de quique que sincroniza com a animação da bola a tocar no chão
- Variação de pitch aleatória $\pm 5\%$ nos sons para evitar repetição mecânica

■ FASE 8 — HUD, Polish e Finalizações

Tarefa 8.1 — HUD Completo

- Pontuação total no topo ao centro com fonte grande e legível
- Estatísticas no canto superior: X/Y cestos, X% FG, X de 3pts
- Indicador de zona actual: 'ZONA DE 3 PONTOS' ou 'ZONA DE 2 PONTOS' actualizado em tempo real
- Minimap opcional no canto inferior: vista de cima do campo com dot a indicar posição do jogador
- Painel de controlos toggle com tecla H: mostra WASD, espaço, etc.

Tarefa 8.2 — Recuperação da Bola

- Após lançamento: bola simulada fisicamente até velocidade < 0.5 m/s ou passados 8 segundos
- Trigger de reset: animar bola a voar de volta para as mãos do jogador (tween em arco curto)
- Alternativa rápida: fade out da bola + fade in nas mãos do jogador após 2s do lançamento
- Estado volta a DRIBBLING após recuperação da bola
- Tecla R como reset manual para casos em que a bola fica presa ou sai dos limites

Tarefa 8.3 — Câmara de Replay de Cesto (Opcional)

- Ao fazer cesto: guardar os últimos 2s de posição da bola num buffer circular
- Câmara corta para uma posição fixa ao lado do cesto por ~2 segundos
- Mostrar a bola a entrar com câmara mais próxima e ângulo dramático
- Retomar câmara de 3ª pessoa após o replay com transição suave

Tarefa 8.4 — Optimizações e Entrega

- Garantir 60fps em hardware mediano: usar geometrias com segmentos razoáveis (não excessivos)
- Frustum culling activo (default no Three.js) — garantir que objectos têm boundingSphere correcto
- Disponibilizar o projecto numa página web estática (GitHub Pages ou similar)
- Código self-contained: todas as texturas em base64 ou carregadas de CDN, sem dependências locais
- Referenciar no código todas as bibliotecas e assets externos utilizados
- Reconhecer uso de ferramentas de IA no comentário inicial do main.js e nos slides

Ordem Sugerida de Implementação

Fase	Tarefas	Prioridade	Dependências
Fase 1	1.1 → 1.2	■ Crítica	Nenhuma
Fase 2	2.1 → 2.2 → 2.3	■ Crítica	Fase 1
Fase 3	3.1 → 3.2 → 3.3	■ Crítica	Fase 1
Fase 4	4.1 → 4.2	■ Alta	Fase 3
Fase 5	5.1 → 5.2 → 5.3 → 5.4	■ Crítica	Fases 1, 2

Fase 6	6.1 → 6.2 → 6.3 → 6.4 → 6.5	■ Crítica	Fases 3, 4, 5
Fase 7	7.1 → 7.2 → 7.3	■ Crítica	Fases 5, 6
Fase 8	8.1 → 8.2 → 8.3 → 8.4	■ Normal	Todas as anteriores

Dicas para Trabalhar com uma LLM

- Pede uma tarefa de cada vez — cola sempre o código actual completo como contexto antes de pedir a próxima tarefa.
- Começa a Fase 6.3 com testes isolados (sem Three.js): calcula a trajetória num script Node.js simples para validar a fórmula.
- A Tarefa 5.3 (aro físico com segmentos) deve ser testada isoladamente antes de integrar o lançamento.
- Guarda versões funcionais do código antes de cada nova fase — mais fácil reverter se algo quebrar.
- Se o jogo ficar lento, reduz o número de shadow-casting lights e o tamanho dos shadow maps.